VIEWS

Delivery Note

Eight HydraNet Forecast Models, Collapsed Three Ways

— ◇ —

Identifier	`hydranet-pgm-global-calibration-2026-09`
Checksum	`36172716`
Date	11-10-2026
Author	Simon Polichinel von der Maase

Scope *what the data is*

Partition	Calibration
Spatial resolution	PRIO-GRID cell
Spatial extent	Global land — 64,818 cells
Temporal resolution	Month
Temporal extent	Forecasts Jan 2018 to Dec 2021; trained on Jan 1990 to Dec 2017
Forecast structure	13 rolling origins, each forecasting 36 months, collapsed three ways
Targets	State-based, non-state and one-sided fatalities

Provenance *what produced it*

Model library	views-hydranet, pinned `~=0.1.1`; exact version not recorded
Framework	views-pipeline-core, version not recorded
Data backend	views-datafactory, pinned `>=1.9.0,<2.0.0`; exact version not recorded
Source commit	`views-models@1b9cebc`
Computed on	Rented cloud GPUs (RunPod), NVIDIA RTX PRO 4500

PRIO – Oslo


VIEWS
PREDICTING CONFLICT &
HUMANITARIAN IMPACTS
PRIO

1 Context

This Delivery Note accompanies `hydranet-pgm-global-calibration-2026-09`, a set of conflict forecasts produced by eight HydraNet models on the calibration partition at global PRIO-GRID month, each collapsed from its posterior draws three different ways. It is the authoritative description of what was handed over. The folder was uploaded on 29 September 2026 with a short README and an audit notebook and no note; this document supplies the note, and supersedes that README where the two disagree.

The purpose of this note is to:

- record what was delivered, in enough detail to identify it again later;
- describe the models that produced it, and what distinguishes them from one another;
- explain why there are three files per model per origin rather than one, and what each one is;
- state how the files are organised and how they are intended to be consumed; and
- set out what is *not* in the delivery, and what is not yet known about it.

This note covers:

- the architecture all eight share, and the eight configurations that differ (Sections 2–3);
- the three point estimators, the arithmetic of each, and which one the models declare (Section 4);
- the file layout, schema and units of the delivered forecasts (Section 5);
- measured magnitude and horizon behaviour — diagnostics, not an evaluation (Section 6); and
- what the delivery excludes, and the caveats that bear on using it (Sections 7–8).

This note explicitly does *not* cover:

- **any evaluation against observed outcomes.** Section 6 reports what the models predict and how that varies by forecast step and by estimator. It scores nothing, ranks nothing and compares nothing to actuals beyond the single magnitude ratio the run itself computed;
- any recommendation as to which of the eight, or which of the three estimators, to prefer. That is what the selection routine is for;
- the validation or forecasting partitions, or any live forecast; and
- the internal machinery that produced the run — the cost, the hardware and what the deployment cost us are recorded separately, in `reports/runpod_cost_and_time_note_2026-09.md` and `reports/postmortem_runpod_first_deployment_2026-09.md`.

The delivery is a fixed artefact. It is identified by the identifier and checksum on the title page — quote both — and is not revised in place: any correction is issued as a new delivery with a new identifier and a new checksum.

2 The Model

All eight delivered models are the same architecture under eight configurations. The architecture is **HydraNet**, designed and built in-house — the class the eight instantiate is `HydraBNUNet06_LSTM4` in `views-hydranet`. It is my own work: the design, the implementation and the paper describing it (von der Maase, 2026) are mine. It is not a library model and has no upstream.

What It Is

HydraNet treats the global grid as an image sequence rather than as a panel of independent cells. Each training example is a 32-month window of the whole 360×720 half-degree grid, with three input channels — the state-based, non-state and one-sided fatality histories. The network is a recurrent U-Net: two encoder levels with batch normalisation and max pooling compress the grid spatially, four parallel ConvLSTM cells carry temporal memory between months, and skip connections return the compressed representation to full resolution. One set of parameters is shared across every cell, so a cell with almost no conflict history is forecast using structure learned from the whole grid and from its neighbours.

Six Heads, Two Per Target

The name is the architecture's main feature. There are six independent decoder heads — two for each of the three outcomes:

- a **gate** head, trained as a classifier on whether the outcome is non-zero at all, emitting $P(y > 0)$ per cell per month; and
- a **body** head, trained as a regression on how large the outcome is, emitting the parameters of a count distribution rather than a single number.

The split matters because the overwhelming majority of cell-months carry no fatalities at all — in the delivered forecasts, 99.4% of them (Section 6). A single head asked to produce both the zeros and the magnitudes is pulled toward predicting almost nothing everywhere; separating the two lets each be trained on the question it is actually answering. The gate is trained with a weighted binary cross-entropy (positive class weighted 2.0); the body with mean squared error on the $\log(1 + y)$ scale.

A forecast is the two composed. Two compositions are used across the eight, and the choice is the single most consequential setting that varies between them:

- **soft gate** — for each draw, a Bernoulli trial on the gate probability decides whether that cell is non-zero in that draw. A cell with a gate of 0.3 is zero in roughly seven draws out of ten and carries the body's magnitude in the other three.

- **threshold gate** — the cell is kept if the gate exceeds a fixed threshold τ and zeroed otherwise, identically in every draw. Three of the eight use this, at $\tau = 0.14$, 0.16 and 0.20.

The masks are exactly zero or one and are applied on the $\log(1 + y)$ scale, where $\log(1 + 0) = 0$, so the multiplication introduces no approximation.

Why There Are Draws At All

The body head does not emit a number, it emits a distribution — a negative binomial, or a mixture of two negative binomials, depending on the model. Drawing from it gives one possible future; drawing repeatedly gives a sample of them. This delivery carries **16 draws per cell per month**, built as four passes with dropout left active at prediction time, each yielding four draws from the distribution the body emitted ($4 \times 4 = 16$). Both sources of variation are deliberate: the dropout passes express uncertainty about the model, the family draws express uncertainty about the outcome given the model.

Sixteen is a setting for this run and not a property of the architecture. The platform standard is higher; the full-depth configuration peaks at roughly 28.6 GB and does not fit the hardware this run had. The consequence that bears on the delivered files is in Section 4: an upper quantile of 16 draws is a coarse statistic.

What Reaches Disk

The forecasts are produced one month at a time, each month's prediction feeding back as the next month's input, out to 36 months. Because every model declares `evaluation_mode: stochastic`, the run keeps the full set of draws rather than reducing them, and the collapse to a single number per cell happens afterwards and outside the run — which is what Section 4 describes. The draws are composed on the $\log(1 + y)$ scale and are on the count scale by the time they reach the collapse, which refuses them if they are not.

3 The Eight Models

Nearly everything is held fixed across the eight. All of them are `HydraBNUNet06_LSTM4`; all train for 300 lessons of three windows each at learning rate 0.001 with the same warm-up and decay schedule; all take the same three input channels on the same global land grid of 64,818 cells; all target the same three outcomes on the $\log(1 + y)$ scale; all produce 16 draws; all were trained in the same two days on the same rented hardware from the same commit. Those facts are stated once here and are not repeated per model.

The link columns open each model's configuration on GitHub, fixed at the delivery commit on the title page: **Config** is `config_hyperparameters.py`, which sets the body family, the composition, the dropout rate and the training schedule; **Feat.** is `config_queryset.py`, which declares the inputs. The names are arbitrary labels assigned by the `adjective_noun` convention used across views-models and carry no meaning.

Table 1: The eight delivered models, and the five settings that differ between them.

Model	Body family	Composition	Window sampling	Scheduled sampling	Config	Feat.
purple_alien	mixture NB	soft gate	threshold	off	link	link
pink_pirate	mixture NB	soft gate	threshold	off	link	link
blue_stranger	mixture NB	soft gate	Boltzmann	off	link	link
bold_comet	mixture NB	threshold, 0.14	threshold	off	link	link
blazing_meteor	mixture NB	threshold, 0.16	threshold	off	link	link
heavy_freighter	NB	soft gate	threshold	**on**	link	link
bright_starship	NB	soft gate	threshold	off	link	link
violet_visitor	NB	threshold, 0.20	sigmoid	off	link	link

Two further settings vary and are not in the table because they are not interpretable as treatments: the random seed (42–47, locked and logged per run) and the dropout rate (0.125 in six models, 0.1 in `blue_stranger`, 0.15 in `violet_visitor`).

The Eight, One Paragraph Each

`purple_alien` — Mixture-negative-binomial body, soft gate. The reference configuration the other seven are variations on, and the only one carrying bespoke regression-loss weights. It is the model used for the worked examples throughout this note.

`pink_pirate` — Identical to `purple_alien` in body family and composition, with the standard regression-loss weights and a different seed. The nearest thing in the set to a seed replicate of the reference configuration.

`blue_stranger` — Mixture-negative-binomial body, soft gate, and the only model whose training windows are drawn by Boltzmann weighting: a window's anchor cell is sampled with probability proportional to $\exp(\text{activity}/\tau)$ rather than uniformly from cells above an activity threshold. It also carries the lowest dropout rate of the eight, 0.1.

`bold_comet` — Mixture-negative-binomial body with a threshold gate at $\tau = 0.14$: a cell is forecast as non-zero only where the gate exceeds that probability, and identically in all 16 draws. The lowest of the three thresholds, so the most permissive of the three threshold-gate models.

`blazing_meteor` — As `bold_comet` with the threshold raised to $\tau = 0.16$. The pair is the closest thing in the set to a clean reading of what moving the threshold does, though the seeds differ between them.

`heavy_freighter` — Single-negative-binomial body, soft gate, and **the only model trained with scheduled sampling active**, which feeds the model's own predictions back during training rather than always the observed history. It is retained deliberately: it was the only configuration predicting fatalities at a realistic scale, and it remains the highest-magnitude model of the eight (Section 6).

`bright_starship` — Single-negative-binomial body, soft gate, scheduled sampling off. Intended

as the controlled contrast to `heavy_freighter`, and the lowest-magnitude model of the eight. **The contrast is confounded:** the two also differ in seed, so the gap between them cannot be attributed to scheduled sampling alone.

`violet_visitor` — Single-negative-binomial body with the highest threshold gate, $\tau = 0.20$, the highest dropout rate, 0.15, and the only sigmoid window sampler, which weights a window's anchor cell by $\sigma(k(\text{activity} - \text{threshold}))$ instead of applying a hard cut.

What the Design Does and Does Not Establish

The two axes that were chosen as treatments are the body family (mixture of two negative binomials in five models, a single negative binomial in three) and the composition (soft gate in five, threshold gate in three). Both are real trades rather than one option dominating: the threshold gate ranks slightly better on the run's own metrics and predicts measurably less on every model it was tried on, which is why both are carried into the selection rather than one being chosen here.

Three things about this set are assumptions rather than findings, and are recorded as such: the five-to-three split between the two body families, the specific seeds, and the premise that eight separate trainings are more informative than fewer models each carrying more compositions. The eight are a spread of plausible configurations submitted for selection, not a factorial design from which an effect can be read.

4 The Three Frames

There are three files for every model and every forecast origin. **They are not three models and not three runs.** There was one run per model. It produced 16 draws for every cell and every month, and a point forecast needs one number rather than 16, so something has to collapse them. There is more than one defensible way to do that, and three were used.

Table 2: The three frames. One run, one set of draws, collapsed three ways.

Frame	Directory	What it is
I	`I_mean`	The arithmetic mean of the 16 draws, taken in double precision on the count scale. **This is the estimator the models declare**, and the only one of the three that the delivery pipeline produces.
II	`II_q95`	The 95th percentile across the 16 draws. An upper estimate: the magnitude that is exceeded in roughly one draw in twenty.
III	`III_conditional_mean`	$\mathrm{E}[y \mid y > 0]$ — the mean taken over the positive draws only, equivalently $\mathrm{E}[y]/P(y > 0)$. The expected magnitude *given* that something happens, rather than the expected magnitude including the chance that nothing does.

Lexical sorting puts these in the order III, II, I, so a file browser or a Drive listing will show them backwards. The Roman numerals are in the directory names so that the intended reading order

survives being sorted.

Why the Collapse Happens Outside the Run

All eight models declare `evaluation_mode: stochastic`, which tells the run to keep the full set of draws and not to reduce them. Each model also declares an aggregation method (`arithmetic_mean`), but under stochastic evaluation that declaration has no effect and the run says so in its own log. The consequence is that the collapse is a separate, later step, taken deliberately rather than as a side effect of how the run was configured — and that is the step that produced these three frames.

What Frame I Can Be Checked Against, and What Frames II and III Cannot

This needs stating plainly, because it is the one asymmetry in the delivery.

Frame I is reproducible. It is produced by a collapse step held in views-models, at `tools/collapse/collapse_predictions.py`, which implements the arithmetic mean and the median and refuses anything else. It takes the mean in double precision, on raw counts, and refuses the input outright on a non-finite value, a negative value, a disagreement in draw count between targets, a duplicated cell-month, or a maximum low enough to suggest the values are still on a log scale. Those refusals are tested.

Frames II and III are not reproducible, and no code for them exists. Neither the percentile nor the conditional mean is implemented anywhere in views-models; both were computed once, in an ad-hoc step, and the posterior draws they were computed from are not on any machine we still have. The draws were written and compressed on the rented GPUs — the run manifests record 13 GB of raw draws per model compressing to roughly 9 MB — but they were never transferred off, and the machines are gone. **So frames II and III cannot be recomputed, re-derived or checked against the posterior.** They are what they are: a probe, recorded here as such.

This also corrects a claim made in my own cost memo of 28 September, which stated that the full posterior had been retained and that a different summary statistic could therefore be produced without a new run. That is not true on disk. **Any statistic other than these three needs a new GPU run.**

Two Properties of These Estimators

Frame II is coarse at 16 draws. The 95th percentile of 16 values falls between the 15th and 16th order statistic, so a single draw moves it, and at a sample this small an upper quantile is biased low. It is reported as computed, with no small-sample correction. If the draw count is ever raised, frame II must be re-measured before anything relies on it.

Frame III inflates the sparse targets most. Dividing by $P(y > 0)$ multiplies a cell that was positive in one draw of 16 by about 16. The effect is largest where positive draws are rarest,

which is why frame III runs 4.2 times frame I on state-based fatalities but 7.5 times on one-sided (Section 6). One ratio quoted for all three targets would hide that.

How the Three Are Meant to Be Used

All three go into the selection routine as if they were separate models, and the published selection criteria decide. That is settled. I am not proposing one of them here, and the frames are not ranked anywhere in this note.

One provenance correction while the frames are in view: the README uploaded with the folder cites "ADR-023 Amendment 1" as the authority for the three frames. No such document exists. ADR-023 governs the collapse and was accepted on 28 September 2026; its only amendment is dated 6 October 2026 and concerns an unrelated library. The authority for frame I is ADR-023 and the commit on the title page. Frames II and III have no governing document, which is part of why this section says what it says about them.

5 The Delivered Files

The delivery is one folder per model, eight in all, each holding three frame directories of thirteen Parquet files:

```
<model>/
  I_mean/                 predictions_calibration_<ts>_<NN>.parquet  (13 files)
  II_q95/                 "
  III_conditional_mean/   "
```

That is $8 \times 3 \times 13 = 312$ files and a little over 180 MB. The timestamp in each filename is the model's own run, so it is the same within a model and differs between models; the two-digit suffix `_00` to `_12` is the forecast origin.

Parquet is a columnar table format. It is not a text format and will not open in a text editor, but every analysis environment reads it directly, with no conversion step:

Python

```
import pandas as pd
f = "purple_alien/I_mean/predictions_calibration_20260928_201214_00.parquet"
df = pd.read_parquet(f)
```

R

```
library(arrow)
f <- "purple_alien/I_mean/predictions_calibration_20260928_201214_00.parquet"
df <- read_parquet(f)
```

Thirteen Origins, and Why They Must Not Be Stacked

The calibration test window is months 457 to 504 — January 2018 to December 2021. Each model forecasts 36 months ahead from each of thirteen successive starting points, one month

apart.

Table 3: The thirteen forecast origins. Each file covers 36 months from its own origin.

File	Forecast from	Months covered	Calendar
`_00`	month 456	457–492	Jan 2018 – Dec 2020
`_01`	month 457	458–493	Feb 2018 – Jan 2021
`_02`	month 458	459–494	Mar 2018 – Feb 2021
. . . one month later at each step . . .			
`_11`	month 467	468–503	Dec 2018 – Nov 2021
`_12`	month 468	469–504	Jan 2019 – Dec 2021

The windows overlap, so the same cell-month appears in many files. Month 480 appears in all thirteen, forecast from thirteen different distances. Concatenating the thirteen files and treating the result as a panel produces a table with up to thirteen different values for the same cell and month, which is not a forecast of anything. Pick one origin, or carry the origin as an explicit dimension. The audit notebook in the folder prints the overlap as a table if it helps to see it.

Schema

Every one of the 312 files holds exactly five columns and **2,333,448 rows** — 64,818 cells × 36 months, a complete block with no gaps. The schema is byte-identical across all 312.

Table 4: The five columns, identical in every file.

Column	Type	Meaning
`month_id`	integer	VIEWS month index; 457 is January 2018
`priogrid_id`	integer	PRIO-GRID cell identifier (see below)
`pred_lr_sb_best`	double	predicted state-based fatalities
`pred_lr_ns_best`	double	predicted non-state fatalities
`pred_lr_os_best`	double	predicted one-sided fatalities

`priogrid_id` is a cell, not a coordinate. PRIO-GRID is a 720 × 360 grid of half-degree cells covering the globe, numbered from 1 in the south-west corner and running east before north. To place a row on a map:

Table 5: Converting `priogrid_id` to coordinates. The result is the cell centre.

Row (south to north)	$\texttt{row} = \lfloor(\texttt{priogrid_id} - 1)/720\rfloor$
Column (west to east)	$\texttt{col} = (\texttt{priogrid_id} - 1) \bmod 720$
Latitude	$-90 + (\texttt{row} + 0.5) \times 0.5$
Longitude	$-180 + (\texttt{col} + 0.5) \times 0.5$

In R:

```
d$lat <- -90 + (((d$priogrid_id - 1) %/% 720) + 0.5) * 0.5
d$lon <- -180 + (((d$priogrid_id - 1) %% 720) + 0.5) * 0.5
```

As a check: in `purple_alien`'s first origin the largest predicted state-based value is 58.0 fatalities in month 462 — June 2018 — at `priogrid_id` 181875, which resolves to 36.25°N, 37.25°E, in northern Syria. The 64,818 cells delivered here are the land subset of the 259,200 in the full grid; their centres span 55.75°S to 83.25°N.

Values are fatality counts. They are continuous rather than integer, because a mean over draws is not a whole number, and they are on the original count scale — not logged, not scaled, not truncated. Observed values are not included (Section 7).

Checking the Delivery Without Running Anything

The folder contains `audit_selector_frames.ipynb`, which audits the folder it sits in. It opens all 312 files, checks the columns and types, checks for non-finite, negative and duplicated values, verifies that every file is a complete 64,818 × 36 block, draws the global maps and the per-step profiles, and ends in a cell that *raises* rather than prints if anything is wrong.

Its outputs are saved from the run that checked this exact copy of the data, so it can be read without executing it. To re-run it, start Jupyter in the folder and Run All; it needs only `pandas`, `pyarrow`, `numpy` and `matplotlib`, and reads the model directories beside it. Nothing else in this delivery needs to be verified by hand, and there is no checksum file to run.

One gap to record: the delivered folder carries **no per-model manifest.** The engine version, run host, runtime and configuration commit for each model exist, in the run records held in views-models, and the commit is on the title page of this note — but they do not travel inside the folder. A later delivery of this kind should ship them beside the data.

6 Magnitude and Horizon

This section ranks nothing and recommends nothing. It reports what the delivered files contain and how that varies by model, by estimator and by forecast step. One quantity here is measured against observed outcomes — the magnitude calibration ratio, which the run itself computed. Everything else is a property of the predictions alone. It is here because these three behaviours determine whether a given file is being read for the thing it can support, and none of them is visible from the files without computing it.

Magnitude: All Eight Under-Predict, By Factors That Differ Fourfold

The magnitude calibration ratio is MCR = $\bar{\hat{y}}/\bar{y}$, the mean prediction over the mean observed value on the same cell-months. One is perfect aggregate calibration, below one is systematic under-prediction. Because it averages over the draws before dividing, it is exactly the ratio frame I's values bear to the observed totals.

Two things in that table are worth stating rather than leaving to be noticed.

Every model under-predicts magnitude, and frame I under-predicts it by between four and fifteen times. The least timid of the eight reaches a quarter of the observed total;

Table 6: Magnitude calibration and CRPS on the calibration test window, as computed by the run. Ordered by state-based MCR. Lower CRPS is better; MCR nearer 1 is better calibrated.

Model	MCR sb	MCR ns	MCR os	CRPS sb
bright_starship	0.067	0.057	0.059	0.1143
blue_stranger	0.073	0.063	0.040	0.1136
purple_alien	0.093	0.127	0.095	0.1130
violet_visitor	0.110	0.169	0.106	0.1132
pink_pirate	0.195	0.135	0.060	0.1115
heavy_freighter	0.215	0.303	0.241	0.1135
bold_comet	0.240	0.171	0.257	0.1107
blazing_meteor	0.250	0.319	0.328	0.1101
range	*3.8×*	*5.6×*	*8.2×*	*3.8%*

the most timid reaches a fifteenth. The shortfall is in severity rather than in location: the models mark roughly the right places and put numbers that are too small in them. This is a property of the estimator and the objective, not of this export — a body trained on mean squared error over a field that is 99.4% zeros is rewarded for staying near zero.

CRPS does not separate the eight the way magnitude does. Across the eight it spans 3.8% while state-based magnitude spans 3.8-fold. The two orderings also run opposite ways here: the most timid model carries the worst CRPS and the least timid the best. A 3.8% spread is a narrow basis for a decision, and the selection criteria — not this note — determine what weight it carries.

The Three Estimators, Per Target

Totals over all eight models and all thirteen origins, with each frame's ratio to frame I.

Table 7: Total predicted fatalities per frame and target, summed over 8 models × 13 origins, with the ratio to frame I.

	I: mean	II: q95		III: E[y|y>0]	
Target	total	total	ratio	total	ratio
State-based	4,194,894	18,898,471	4.51×	17,815,285	4.25×
Non-state	731,151	3,522,070	4.82×	4,417,776	6.04×
One-sided	427,970	1,979,497	4.63×	3,217,289	**7.52**×
All three	5,354,015	24,400,038	4.56×	25,450,349	4.75×

Frames II and III both land near five times frame I in total, which is close to the factor by which frame I falls short of observed — that is what makes them interesting and is the reason all three were kept. But the per-target columns are the point of the table: frame III runs 4.25 times frame I on state-based fatalities and 7.52 times on one-sided, for the reason given in Section 4. A single ratio quoted for all three targets would be wrong for two of them.

All three frames are non-zero on exactly the same rows. 1,469,004 of the 242,678,592

rows in each frame carry a non-zero prediction — 0.61%, so 99.4% of the delivered field is exact zeros. Those rows fall on 98,995 distinct cell-months and 6,552 distinct PRIO-GRID cells, a tenth of the 64,818 delivered. The three estimators differ in magnitude and never in where conflict is predicted; this was verified across all 312 files, not sampled. Note that the figure 1,469,004 counts *rows*, which means the same cell-month is counted once per model and once per origin; it is not a count of distinct cell-months.

Horizon: The Field Halves Over Five Steps, Then Stops Changing

Total predicted state-based fatalities by forecast step, averaged over the thirteen origins and summed over the eight models.

Table 8: Predicted state-based fatalities by forecast step. Frame I's step-1 value is 100%.

Step	I: mean	II: q95	III: E[y|y>0]	I as % of step 1
1	16,404	71,160	56,415	100.0%
2	11,883	52,554	46,616	72.4%
3	10,058	44,794	41,453	61.3%
4	9,455	42,590	39,164	57.6%
5	8,962	40,480	38,155	54.6%
6	8,789	39,563	37,213	53.6%
9	8,468	38,259	36,754	51.6%
12	8,393	37,860	35,606	51.2%
18	8,674	39,266	37,939	52.9%
24	8,607	38,698	36,794	52.5%
30	8,480	38,353	36,287	51.7%
36	8,728	39,592	37,336	53.2%

The informative part of these forecasts is the first few months. Between step 1 and step 5 the predicted field falls by 45%. From step 6 to step 36 — thirty-one of the thirty-six months delivered — frame I stays between 8,265 and 8,907, a spread of 7.8% with no trend, and the other two frames behave the same way. The models carry little dynamic signal past about five months; what they emit after that is close to a constant field.

The practical consequence: **a difference between two of these models at month 30 is not a difference in forecast, and reading month 30 as a forecast is a misuse of the file.** Any use that weights the 36 months equally — including a metric averaged over the horizon — is dominated by steps at which the models are no longer forecasting.

Two things this measurement does not establish. It is the *predicted* total, not a comparison to observed, so it says the model stops responding and not that it becomes wrong. And an earlier scrutiny of these eight on a smaller region and a shorter training budget found per-step magnitude decaying toward zero rather than flattening; that was a different run and this measurement supersedes it for these files, but the two are different quantities and neither should be read into the other.

7 Not Included in This Delivery

Table 9: What this delivery does not contain, and where it is instead.

Not included	Where it is, or why not
Observed values	The files carry predictions only. Actuals come from the data store, keyed on the same `month_id` and `priogrid_id`, so a join needs nothing from here.
The posterior draws	Produced and compressed on the rented machines, never transferred off them, and the machines are gone. This is the one omission with consequences — Section 4.
Evaluation metrics	Beyond the magnitude and CRPS figures in Section 6, which the run computed on its own output, nothing has been scored. The per-step and per-origin evaluation outputs were written on the machines and did not come home either.
Per-model manifests	No provenance file travels inside the folder. Run host, runtime, engine version and configuration commit are held in views-models; the commit is on the title page.
Training logs	Held in views-models beside each model's output, not shipped. They carry the metric tables Section 6 quotes.
Other partitions	Calibration only. No validation partition, no forecasting partition, no live forecast.
Model artefacts	No trained weights. None from this run exist on any machine we hold, so the run cannot be resumed or re-predicted from — only repeated from the configuration.

8 Caveats and Notes

Nothing here has been evaluated in the sense the selection routine means. The figures in Section 6 are the ones the run computed on its own output, plus three measurements taken from the delivered files for this note. No model has been scored against the published criteria, ranked against another, or compared with any model outside these eight.

Two of the three frames cannot be reproduced. Frame I is produced by code in views-models and its refusals are tested. Frames II and III were computed once, outside that code, from draws that were never transferred off the rented machines. They cannot be recomputed or checked, and no governing document describes them. Section 4 sets this out in full; it is repeated here because it is the caveat most likely to matter later.

A statistic other than these three needs a new run. My cost memo of 28 September said the full posterior had come home and that a different summary form could be produced without paying for more compute. That was wrong: the draws are not on disk. Anything else — a different quantile, a different conditional, a predictive interval — requires training to be repeated on rented hardware.

The software is pinned as ranges, not versions. The configuration commit on the title page

is exact, and the run logs record the CUDA build. The engine versions are not recorded anywhere: the models pinned `views-hydranet ~=0.1.1` and `views-datafactory>=1.9.0,<2.0.0`, the environment that resolved those pins existed only on the rented machines, and no version line was captured before they were destroyed. So the code is identified to a repository state but not to a release. A later delivery should record the resolved versions in a manifest beside the data.

Pinning the code does not pin the input. The predictors were fetched live from the data store at run time. A re-run from the same commit against a later state of that store can produce different numbers without anything in the configuration having changed. What is fixed is this delivery, by its identifier and checksum.

The horizon behaviour has a mechanism, and it is the gate. The gate probability decays with forecast step — roughly thirtyfold by month 36, in all eight models — and that decay is what flattens the field after about five steps. It is not caused by the choice of regression loss; substituting the loss was tried and changed the decay very little. The predicted magnitude conditional on firing is comparatively stable; what fades is the model's willingness to fire at all.

One comparison in the set looks controlled and is not. The models `heavy_freighter` and `bright_starship` share a body family and a composition and differ in scheduled sampling, which invites reading the gap between them as the effect of scheduled sampling. They also differ in random seed. With one run each, the two explanations cannot be separated.

An earlier scrutiny of these eight reached a harsher verdict, on a different run. In August the same eight were examined at a shorter training budget on Africa and the Middle East only, and that report recommended against promotion on the grounds that seven of the eight collapsed to near-zero magnitude within six months. This delivery is a different run — full training budget, global land — and Section 6 measures the behaviour afresh on the files actually delivered. The August report is superseded for these files and is not superseded as a record of what it measured.

The model carries no covariates. Each model sees the three fatality histories and nothing else — no population, no terrain, no governance indices, no event data beyond the outcomes themselves. These are purely autoregressive spatiotemporal models. Whatever structure they use is learned from the spatial and temporal pattern of past fatalities.

9 Next Steps

Four things follow from this delivery. None of them needs a decision from the reader.

1. **All twenty-four go into the selection routine as separate models.** Eight models times three frames. The published criteria decide which collapse wins, rather than my choosing one here and then defending it. This was settled in September and is restated rather than reopened.

2. **The column names in this delivery differ from the darts delivery's, and the consuming tool must be told which it is reading.** Here the columns are `pred_lr_sb_best`,

`pred_lr_ns_best` and `pred_lr_os_best`; the darts delivery uses `pred_lr_ged_sb` and its siblings. `lr_ged_{sb,ns,os}` is the canonical form and future deliveries will carry it, renamed at send-off rather than in the files on disk. Until then the target column has to be set explicitly wherever these files are read, not inferred from the schema — a tool that guesses will silently read the wrong delivery's convention.

3. **The next delivery of this kind ships its provenance inside the folder.** A manifest per model — resolved engine versions, run host, runtime, configuration commit — and the per-step evaluation outputs retained rather than left on the machine that produced them. Both gaps in this delivery trace to the same cause, which is that nothing was brought down from the rented hardware except the predictions.

4. **The posterior draws are retained on any future run before anything is derived from them.** The reason frames II and III are unverifiable is that the draws were compressed on a machine and then lost with it. The compression ratio is large enough — roughly 9 MB per model — that there was never a cost argument for leaving them behind.

Questions about the data, the models or anything in this note come to me.

References

von der Maase, S. P. (2026). Next-generation conflict forecasting: Unleashing predictive patterns through spatiotemporal learning. Manuscript in preparation.